

# Bioinformatics session

4th annual workshop on  
bioinformatics and variant  
interpretation in InPreD

[https://inpred.github.io/26-09\\_bioinfo\\_ws/](https://inpred.github.io/26-09_bioinfo_ws/)



## 2. Containerization



## A brief history 📖

|             |                                                                                                                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                                                                                                                                                                               |
| <b>1979</b> | <code>chroot</code> system call (changing the root directory of a process and its children to a new location in the filesystem) in Uni V7 considered as beginning of process isolation                        |
| <b>2000</b> | FreeBSD Jails allows administrators to partition FreeBSD computer system into several independent, smaller systems ( <i>jails</i> ) – with the ability to assign IP address for each system and configuration |
| <b>2001</b> | Linux VServer is jail mechanism that can partition resources, similar to FreeBSD Jails                                                                                                                        |

|             |                                                                                                                                                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                                                                                                                                                                                 |
| <b>2004</b> | First public beta of Solaris Containers released - combines system resource controls and boundary separation provided by zones                                                                                  |
| <b>2005</b> | Open Virtuozzo is operating system-level virtualization technology for Linux which uses patched Linux kernel for virtualization, isolation, resource management and checkpointing                               |
| <b>2006</b> | Process Containers, launched by Google, was designed for limiting, accounting and isolating resource usage of collection of processes - renamed to <i>Control Groups (cgroups)</i> and merged into Linux kernel |
| <b>2008</b> | Linux Containers (LXC) was the first, most complete implementation of a Linux container manager - implemented using cgroups and Linux namespaces                                                                |



|             |                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
|             |                                                                                                                                             |
| <b>2011</b> | Warden from Cloud Foundry can isolate environments on any operating system, runs as a daemon and provides API for container management      |
| <b>2013</b> | Let Me Contain That For You (LMCTFY) kicked off as open-source version of Google's container stack - providing Linux application containers |
| <b>2013</b> | Docker emerged                                                                                                                              |
| <b>2016</b> | Singularity (later Apptainer) was released                                                                                                  |

# Docker

## What is it? 🔍

- a technology for packaging an application with all of its dependencies into an independent executable unit (a container)
- the container image can be shipped and run consistently across different computing environments

Which benefits does it provide?



1. Consistency
2. Isolated environments
3. Portability
4. Efficiency

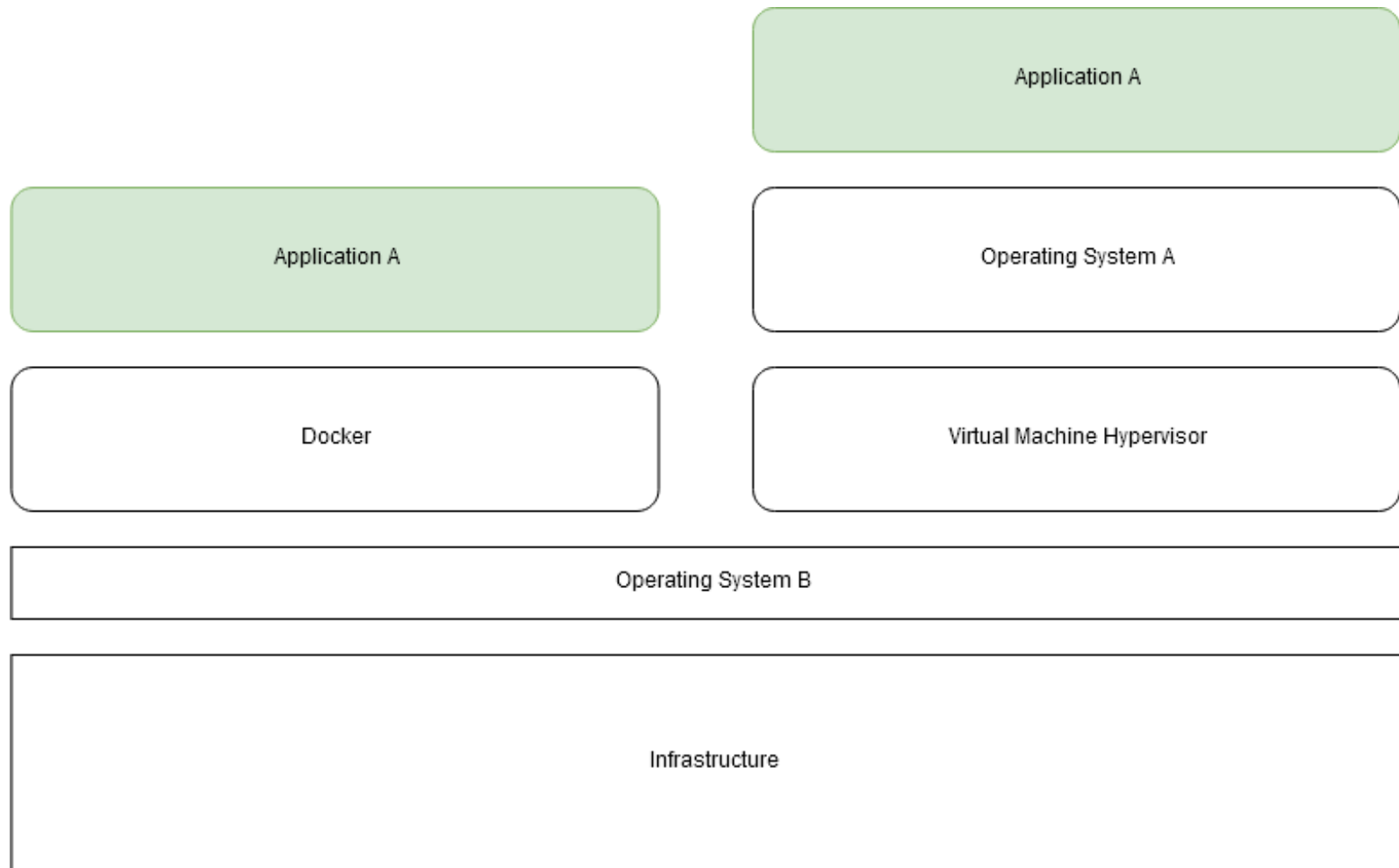
<https://scalablehuman.com/2023/10/24/the-fallacy-of-it-works-on-my-machine>



## How is it different from virtual machines?

- containers use and share host OS's kernel, which makes them more lightweight and efficient
- virtual machines emulate entire physical machines, including the complete OS, making it possible to run multiple OS instances on a single physical machine





<https://dev.to/swikritit/docker-for-dummies-introduction-to-docker-5h67>

## Key Concepts 🔑

1. **Image:** a standardized package that includes all files, binaries, libraries, and configurations necessary to run a given container
2. **Container:** a running instance of an image, operating in an isolated runtime environment
3. **Dockerfile:** instructions on how to build an image
4. **Docker Hub:** a public registry where developers can share their pre-built images

## Key Components 🔑

1. **Docker Engine:** core, open-source technology that builds and runs containers
2. **Docker Client:** command-line tool that sends instructions to the Docker Daemon using REST APIs
3. **Docker Daemon ( `dockerd` ):** receives and processes API requests and calls container runtime
4. **Container Runtime ( `containerd` ):** turns static container image into running, isolated application on host OS; industry standard

# Let's explore! 🌍

Start by going to [https://github.com/InPreD/26-09\\_bioinfo\\_ws\\_docker\\_and\\_ci](https://github.com/InPreD/26-09_bioinfo_ws_docker_and_ci) and create a fork.

The screenshot shows the GitHub interface for the repository '26-09\_bioinfo\_ws\_docker\_and\_ci'. At the top, there are buttons for 'Edit Pins', 'Watch' (0), 'Fork' (0), and 'Star' (0). Below these, a dropdown menu is open, showing the option '+ Create a new fork' highlighted with a red rectangle. The repository is owned by 'marrip' and has 1 branch and 0 tags. The main content area shows a list of files and their commit history:

| File           | Commit Message                  | Time Ago      |
|----------------|---------------------------------|---------------|
| .devcontainer  | chore: add devcontainer setup   | 9 minutes ago |
| src/greeter    | feat: add simple python project | 8 minutes ago |
| .gitignore     | chore: add devcontainer setup   | 9 minutes ago |
| LICENSE        | Initial commit                  | 1 hour ago    |
| README.md      | Initial commit                  | 1 hour ago    |
| pyproject.toml | feat: add simple python project | 8 minutes ago |

Below the file list, there is a section for the README, which includes the title '26-09\_bioinfo\_ws\_docker\_and\_ci' and the description 'codespace template for docker and ci workshop'. On the right side, there are links to 'workshop', 'Readme', 'AGPL-3.0 license', 'Activity', 'Custom properties', '0 stars', '0 watching', '0 forks', 'Audit log', 'Report repository', 'Releases', and 'Packages'.

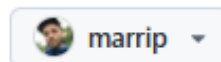
Create your own fork by clicking on **Create fork**.

## Create a new fork

A *fork* is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

*Required fields are marked with an asterisk (\*).*

Owner \*



Repository name \*

/ 26-09\_bioinfo\_ws\_docker\_

✓ 26-09\_bioinfo\_ws\_docker\_and\_ci is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

codespace template for docker and ci workshop

45 / 350 characters

☒ Copy the `main` branch only

Contribute back to InPreD/26-09\_bioinfo\_ws\_docker\_and\_ci by adding your own branch. [Learn more.](#)

You are creating a fork in your personal account.

Create fork



In the forked repository, navigate to **Code** > **Codespaces** > **Create codespace on main**.

The screenshot shows a GitHub repository page for '26-09\_bioinfo\_ws\_docker\_and\_ci'. The repository is public and has 1 branch and 0 tags. The 'Code' button is highlighted with a green background. A dropdown menu is open, showing 'Local' and 'Codespaces' options. The 'Codespaces' option is selected, and a sub-menu is displayed with the text 'No codespaces' and 'You don't have any codespaces with this repository checked out'. A green button labeled 'Create codespace on main' is highlighted with a red border. Below this button is a link 'Learn more about codespaces...'. The repository's README is visible at the bottom, showing the title '26-09\_bioinfo\_ws\_docker\_and\_ci' and the description 'codespace template for docker and ci workshop'.

26-09\_bioinfo\_ws\_docker\_and\_ci Public

Edit Pins Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file <> Code

marrip feat: add simple python project

- .devcontainer chore: add devconta
- src/greeter feat: add simple pyth
- .gitignore chore: add devconta
- LICENSE Initial commit
- README.md Initial commit
- pyproject.toml feat: add simple pyth

Local Codespaces

Codespaces  
Your workspaces in the cloud

No codespaces  
You don't have any codespaces with this repository checked out

Create codespace on main

Learn more about codespaces...

Codespace usage for this repository is paid for by marrip.

README AGPL-3.0 license

## 26-09\_bioinfo\_ws\_docker\_and\_ci

codespace template for docker and ci workshop

About  
codespace template for docker and ci workshop

- Readme
- AGPL-3.0 license
- Activity
- Custom properties
- 0 stars
- 0 watching
- 0 forks
- Audit log
- Report repository

Releases  
No releases published  
[Create a new release](#)

Packages  
No packages published  
[Publish your first package](#)

Inside the terminal, run the following commands:

```
# check for running docker containers
$ docker ps
# check for existing images
$ docker images
# pull image
$ docker pull ubuntu:26.04
# run image
$ docker run ubuntu:26.04
```

Run an interactive container:

```
# start interactive container
$ docker run -it ubuntu:26.04
# print container os version
@ cat /etc/lsb-release
# exit container
@ exit
# print os version
$ lsb_release -a
```

Instruct docker to remove containers after use:

```
# check for any docker containers
$ docker ps -a
# remove all stopped containers
$ docker rm <CONTAINER ID>
# run docker with --rm flag
$ docker run -it --rm ubuntu:26.04
# exit container
@ exit
# check for all docker containers
$ docker ps -a
```

Remove the docker image:

```
# remove docker image
$ docker rmi ubuntu:26.04
# alternatively check for container image id
$ docker images
# and remove the image by using the id
$ docker rmi <ID>
```

## Building an image

We start off by creating a `Dockerfile` in the root directory of our repository ( `/workspaces/26-09_bioinfo_ws_docker_and_ci` ). We add the following to our file:

```
FROM python:3.14-slim-trixie
RUN echo "Hello world!" > greetings.txt
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test cat greetings.txt
```

Let us add a label to our `Dockerfile` to indicate who the author and maintainer is:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
RUN echo "Hello world!" > greetings.txt
```

And then we build and inspect our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker inspect greeter:test
```



Next, we are setting `cat greetings.txt` as a default command that is run whenever the container is started:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
RUN echo "Hello world!" > greetings.txt
CMD ["cat", "greetings.txt"]
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test
```

There is a small python application in this repository that we would like to include in our docker image:

```
FROM python:3.14-slim-trixie
LABEL org.opencontainers.image.authors="martin.rippin@helse-bergen.no"
WORKDIR /usr/src/greeter
COPY pyproject.toml ./
COPY src/ ./src/
RUN pip install .
CMD ["greeter"]
```

And then we build and run our docker image:

```
# build tagged docker image
$ docker build . -t greeter:test
# run tagged docker image
$ docker run --rm greeter:test
```

A full list of `Dockerfile` instructions can be found here:

<https://docs.docker.com/reference/dockerfile#overview>

# Apptainer

## What is it?

- container platform designed for ease-of-use on shared systems and in high performance computing (HPC) environments

## How is it different from Docker?

| Apptainer                                                                      | Docker                                                       |
|--------------------------------------------------------------------------------|--------------------------------------------------------------|
| without root-privileges by default                                             | requires root privileges for most operations                 |
| SIF (Singularity Image Format) – immutable, portable, cryptographically signed | Docker/OCI images – layered file systems, mutable by default |
| can run Docker/OCI images                                                      | cannot run SIF images                                        |

## Let's explore! 🌐

Similar to Docker, Apptainer provides a cli:

```
# check for any apptainer container
$ apptainer instance list -a
# pull image
$ apptainer pull docker://ubuntu:26.04
# check identity
$ whoami
# check identity in container
$ apptainer exec docker://ubuntu:26.04 whoami
```

The container image is saved as a `.sif` -file to the working directory.

## Building an image 🛠️

We are creating a file called `greeter.def` in the root of our repository and add the following lines:

```
Bootstrap: docker
From: python:3.14-slim-trixie

%post
    echo "Hello world!" > /usr/src/greetings.txt
```

And then we build and run our apptainer image:

```
# build apptainer image
$ apptainer build greeter.sif greeter.def
# execute apptainer image
$ apptainer exec greeter.sif cat /usr/src/greetings.txt
```

A full list of apptainer definition file sections can be found here:

[https://apptainer.org/docs/user/main/definition\\_files.html#sections](https://apptainer.org/docs/user/main/definition_files.html#sections)



### **3. Continuous Integration (CI)**



## What is it?

- automating integration of code changes from multiple contributors into single software project
- developers frequently merge code changes into central repository where automated tools are used to assert new code's correctness before integration (test, lint, build)

## **Which benefits does it provide?**

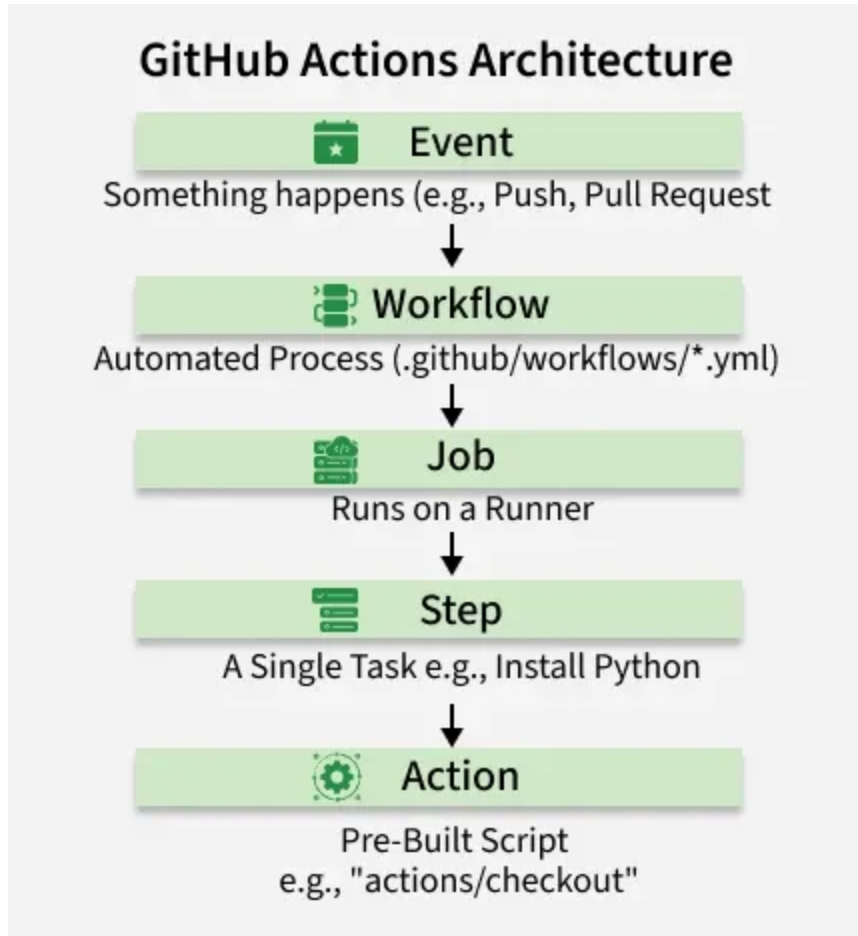
- scale up delivery output
- enables parallel work on features

## GitHub actions

### What is it? 🔍

- GitHub's continuous integration automation platform
- repetitive tasks are triggered based on code pushes, pull requests, or custom schedules

## How does it work? 🤔



<https://www.geeksforgeeks.org/git/introduction-to-github-actions>

## Workflow:

- `.yaml` -file located at `.github/workflows/` containing CI instructions

## Event

- defined in workflow file
- trigger to start workflow, e.g. push, pull\_request, schedule, etc.

## Jobs

- defined in workflow file
- contains several tasks/*steps* and is running on a specific runner (server executing the code of the workflow)
- jobs run in parallel or can depend on each other, e.g. testing before building a container image

## **Step**

- single task inside a job, e.g. installing python

## **GitHub Action**

- reusable steps written by other developers that are freely available
- shorter workflows and avoiding unnecessary repetition

## Example

```
name: Hello World
# event definition
on: [push]

jobs:
  # job definition
  say-hello:
    # runner definition
    runs-on: ubuntu-latest

    steps:
      # github action definition
      - name: Checkout Code
        uses: actions/checkout@v4

      # step definition
      - name: Run a Hello World Script
        run: |
          echo "Hello, world!"
          echo "The current repository branch is ${ github.ref }"
```



**Let's explore!** 🌐

We are adding a workflow file to our repository:

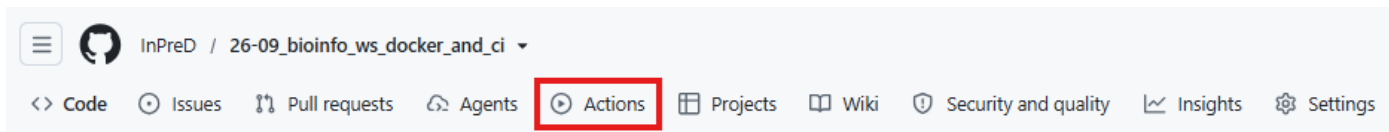
```
# create .github/workflows  
$ mkdir -p .github/workflows
```

Now we add `.github/workflows/hello_world.yaml` using the example file from the previous slide.

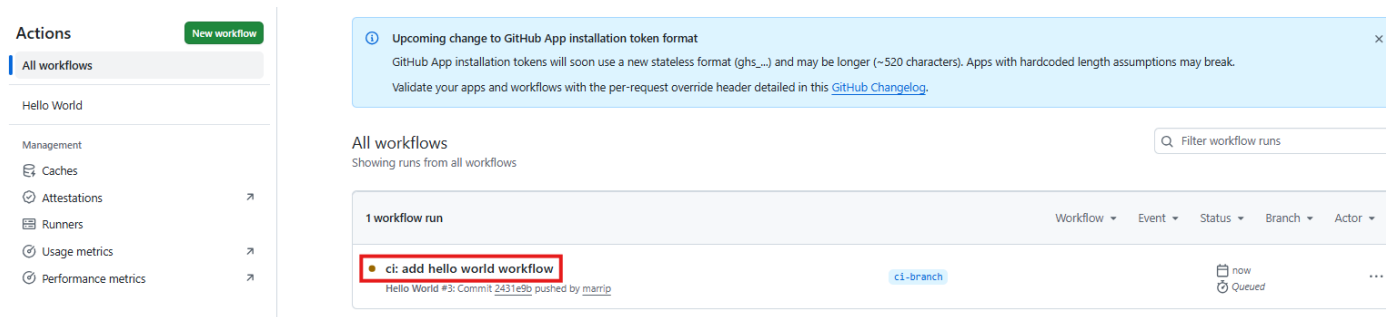
Now we commit out workflow doing:

```
$ git commit -m "ci: add hello world workflow"
```

In the repository on GitHub, we navigate to Actions :



We see that the workflow was triggered by our last commit. To inspect we simple click on the workflow run named by our last commit message.



The screenshot displays the GitHub Actions interface. On the left, the 'Actions' sidebar is visible with a 'New workflow' button and a list of workflow categories: 'All workflows', 'Hello World', 'Management', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main content area shows a notification about an upcoming change to GitHub App installation token format. Below this, the 'All workflows' section is active, displaying a table of workflow runs. The table has columns for Workflow, Event, Status, Branch, and Actor. A single workflow run is listed with the name 'ci: add hello world workflow', which is highlighted with a red box. The branch is 'ci-branch' and the status is 'Queued'.

Actions New workflow

All workflows

Hello World

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs\_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break.

Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

All workflows

Showing runs from all workflows

Filter workflow runs

| Workflow                     | Event | Status | Branch    | Actor |
|------------------------------|-------|--------|-----------|-------|
| ci: add hello world workflow |       | now    | ci-branch |       |

Hello World #3: Commit 2431e9b pushed by marnip

Queued

Our job `say-hello` was triggered. Select the job to inspect it.

← Hello World

✓ ci: add hello world workflow #3

Re-run all jobs

...

Summary

All jobs

say-hello

Run details

Usage

Workflow file

Triggered via push 1 minute ago

mamip pushed

2431e1e

ci-branch

Status

Success

Total duration

7s

Artifacts

—

hello\_world.yaml

on: push

✓ say-hello

3s

🔄

−

+

We see several steps being run which we can expand by clicking on them. But most important, our workflow was successfully run! 🎉

say-hello

succeeded 2 minutes ago in 3s

Search logs

Set up job

15

1

Current runner version: '2.337.0'

2

▶ Runner Image Provisioner

9

▶ Operating System

13

▶ Runner Image

18

▶ GITHUB\_TOKEN Permissions

39

Secret source: Actions

40

Prepare workflow directory

41

Prepare all required actions

42

Getting action download info

43

Download action repository 'actions/checkout@v7' (SHA:3d3c42e5aac5ba805825da76410c181273ba90b1)

44

Complete job name: say-hello

Checkout Code

85

1

▶ Run actions/checkout@v7

17

Syncing repository: InPreD/26-09\_bioinfo\_ws\_docker\_and\_ci

18

▶ Getting Git version info

22

Temporarily overriding HOME='/home/runner/work/\_temp/bb1914fe-6a83-4b02-9151-8ec6d2a100f6' before making global git config changes

23

Adding repository directory to the temporary git global config as a safe directory

24

/usr/bin/git config --global --add safe.directory /home/runner/work/26-09\_bioinfo\_ws\_docker\_and\_ci/26-09\_bioinfo\_ws\_docker\_and\_ci

25

Deleting the contents of '/home/runner/work/26-09\_bioinfo\_ws\_docker\_and\_ci/26-09\_bioinfo\_ws\_docker\_and\_ci'

26

▶ Determining repository object format

27

▶ Initializing the repository

44

▶ Disabling automatic garbage collection

46

▶ Setting up auth

61

▶ Fetching the repository

69

▶ Determining the checkout info

70

/usr/bin/git sparse-checkout disable

71

/usr/bin/git config --local --unset-all extensions.worktreeConfig

72

▶ Checking out the ref

76

/usr/bin/git log -1 --format=%H

77

2431e9b1e654231e21fe2f76d830e80a7c2a73a3

Run a Hello World Script

15

1

▶ Run echo "Hello, world!"

5

Hello, world!

6

The current repository branch is refs/heads/ci-branch

Post Checkout Code

85

Complete job

85